

팍리스 실무형 일경험 프로그램

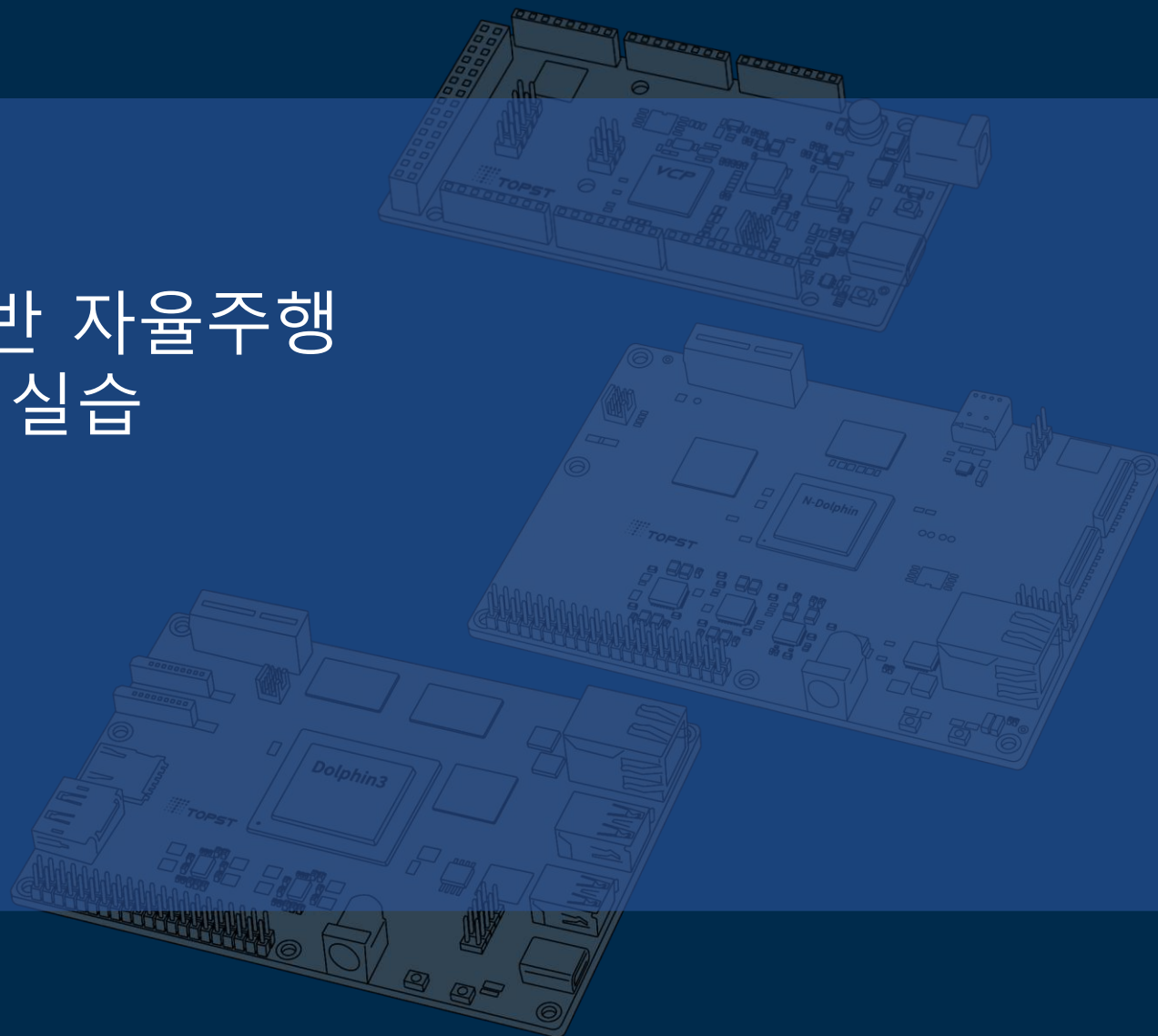
SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



2일차

05 VCP-G ADC 실습

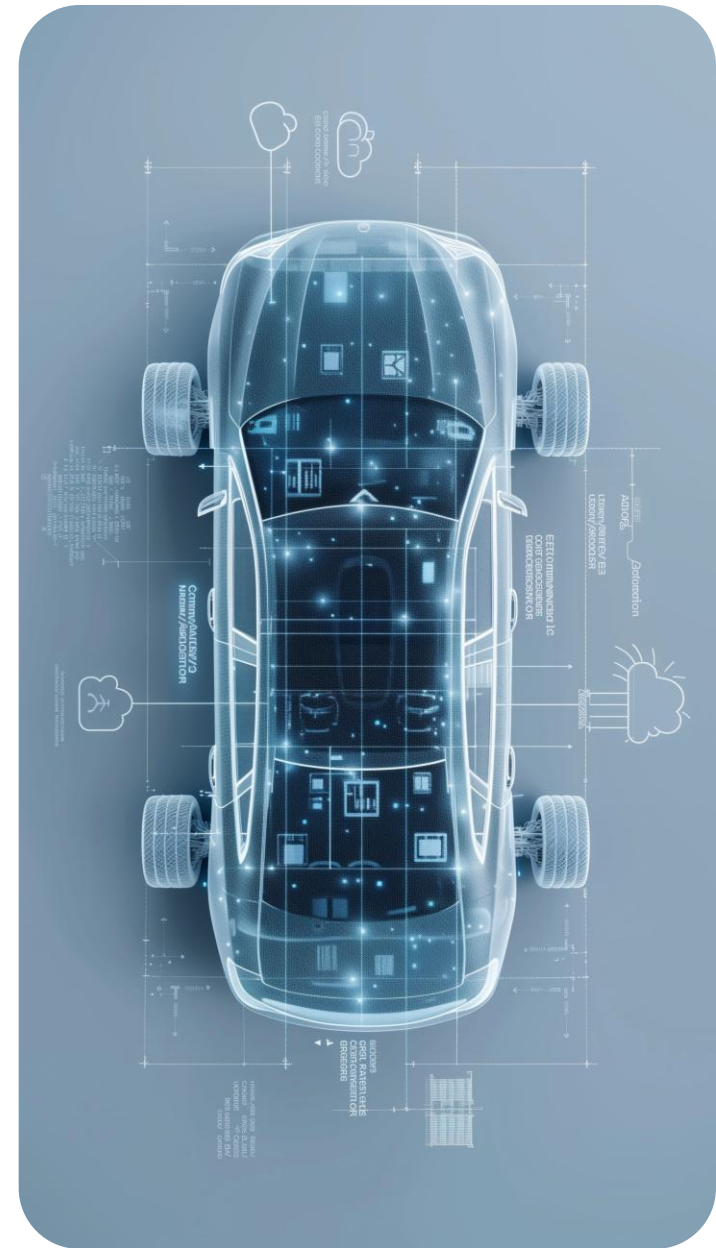
Telechips TOPST



목차

Table of Contents

장비 소개	01
ADC 이해	02
회로 구성 (VCP-G ↔ Bread Board)	03
ADC Control 프로그래밍	04



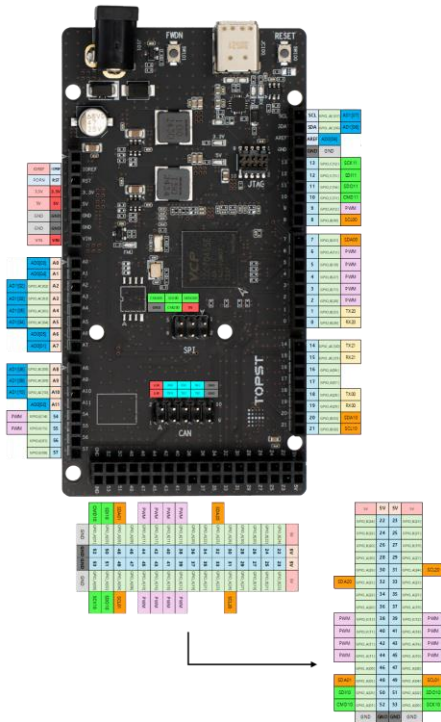
ADC 이해

- **ADC(Analog to Digital Converter)**
 - 연속적인 아날로그 신호를 이산적인 디지털 값으로 변환
 - 센서데이터를 기계가 이해할 수 있도록 변환 필요
 - 변환 과정
 - 1) 샘플링 – 일정 시간 간격으로 아날로그 값을 채취
 - 2) 양자화 – 측정 값을 여러 구간 중 하나로 근사
 - 3) 부호화 – 이 구간을 이진수로 표현
 - 출력 단위 : N비트 분해능일 경우, $2^N - 1$

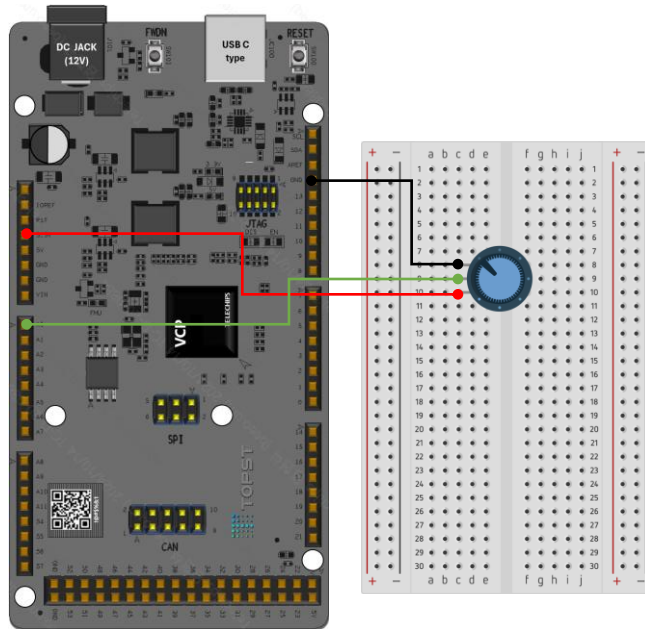


VCP-G 핀맵

- TOPST DOCS에서 VCP-G 핀 FUNC 및 GPIO 번호 확인
 - <https://topst.ai/tech/docs?page=1714a3dcdf19dbf61088d7364d61e29f90a15ff2>



회로 구성 (VCP-G ↔ Bread Board, Potentiometer)



Pin Name	VCP-G Board	GPIO
Potentiometer left	GND	-
Potentiometer Middle	A0	AD0[03]
Potentiometer right	3.3V	-

- Experiment circuit
 - Requirement
 - Breadboard
 - Potentiometer * 1
 - ADC 사용

VIM 사용법



- **VIM 에디터 사용법**

- VIM으로 파일 열기
 - `$ vim ~/test.c`
- 입력 : "i"
- 입력 취소 : "esc 키"
- 저장 후 종료 : ":wq"
- 지우기 : i(insert) 활성화 된 상태에서 "Backspace 키 혹은 Delete 키"

ADC Code

~/topst-vcp/sources/dev.drivers/adc/adc.h에 아래와 같이 명시

```
#define ADC_MODULE_0          (0U)
#define ADC_MODULE_1          (1U)

typedef enum ADCChannel
{
    ADC_CHANNEL_0              = 0,
    ADC_CHANNEL_1              = 1,
    ADC_CHANNEL_2              = 2,
    ADC_CHANNEL_3              = 3,
    ADC_CHANNEL_4              = 4,
    ADC_CHANNEL_5              = 5,
    ADC_CHANNEL_6              = 6,
    ADC_CHANNEL_7              = 7,
    ADC_CHANNEL_8              = 8,
    ADC_CHANNEL_9              = 9,
    ADC_CHANNEL_10             = 10,
    ADC_CHANNEL_11             = 11,
    ADC_CHANNEL_12             = 12,
    ADC_CHANNEL_13             = 13,
    ADC_CHANNEL_MAX            = 14
} ADCChannel_t;
```

AD03핀을 사용할 것이기 때문에, Module은 0, Channel은 3을 사용
(ex, AD14핀 사용은 Module 1, Channel 4 사용)

ADC Code – pot.h / pot.c

~/topst-vcp/sources/app.sample/test.app.adc/pot.h

```
#ifndef MCU_BSP_POT_H
#define MCU_BSP_POT_H

void Potentiometer_Run(void); // main task에서 호출할 진입
함수

#endif
```

~/topst-vcp/sources/app.sample/test.app.adc/pot.c

```
#include <adc.h>
#include <stdint.h>
#include "pot.h"

#define ADC_TYPE          0
#define ADC_MODULE        0 // 사용할 ADC Module 번호
#define ADC_CHANNEL        ADC_CHANNEL_3 // 측정할 채널

void Potentiometer_Run(void)
{
    uint32_t adc_val = 0; // ADC 변환 결과(12bit data)를
저장할 변수

    ADC_Init(ADC_TYPE, ADC_MODULE); // ADC 초기화

    while (1)
    {
        adc_val = ADC_Read(ADC_CHANNEL, ADC_MODULE, 0);
        // ADC 값 읽기
        adc_val &= 0xFFF; // 상위 상태 비트 제거 -> 하위
12bit(실제 ADC 값)만 추출

        mcu_printf("[ADC] CH%d = %4d\n", ADC_CHANNEL,
adc_val); // 결과를 UART 콘솔로 출력

        SAL_TaskSleep(100);
    }
}
```


ADC Code – rules.mk / main.c

~/topst-vcp/sources/app.sample/test.app.adc/rules.mk ~/topst-vcp/sources/app.sample/app.base/main.c

```
SRCS += adc_test.c
```

```
//추가
```

```
SRCS += pot.c
```

```
#include "pot.h" // 변경
```

```
Static void Main_StartTask(void * pArg)
```

```
{
```

```
    (void)pArg;
```

```
    (void)SAL_OsInitFuncs();
```

```
    // 변경
```

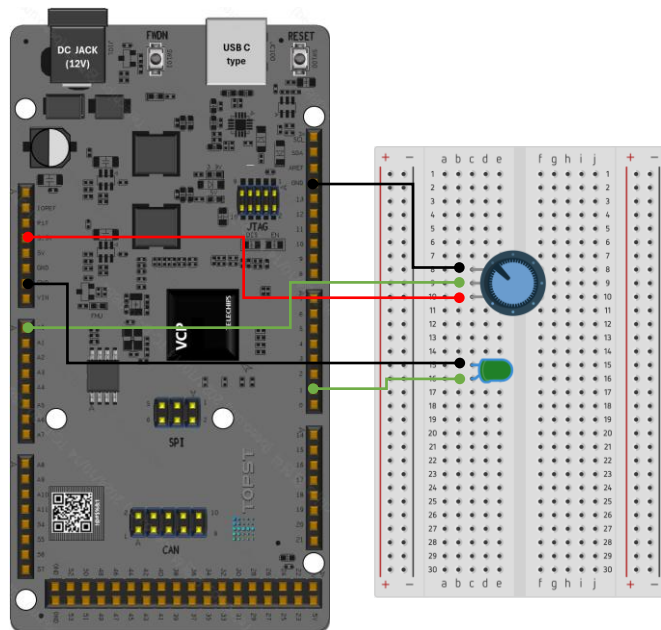
```
    Potentiometer_Run();
```

```
}
```

Build

- build 진행
 - `$ cd ~/topst-vcp/build/tcc70xx/gcc`
 - `$ make clean && make`
- Build 완료 후 FWDN (빌드한 디렉토리에서 바로 실행)
 - `$ sudo ~/topst-vcp/tools/fwdn_vcp/fwdn`
`--fwdn ~/topst-vcp/tools/fwdn_vcp/vcp_fwdn.rom`
`-w ~/topst-vcp/build/tcc70xx/gcc/output/tcc70xx_pflash_boot_2M_ECC.rom`
- `minicom -D /dev/ttyUSB0 -b 115200 -8` 실행 후 보드 재부팅 시 UART 로그에 가변저항 값 출력.

회로 구성 (VCP-G ↔ Bread Board, Potentiometer, LED)



• Experiment circuit

- Requirement
 - Breadboard
 - Potentiometer * 1
 - LED * 1
- ADC 사용

Pin Name	VCP-G Board	GPIO
Potentiometer left	GND	-
Potentiometer Middle	A0	AD0[03]
Potentiometer right	3.3V	-
LED+	1	B[25]
LED -	GND	-

ADC Code

~/topst-vcp/sources/dev.drivers/adc/adc.h에 아래와 같이 명시

```
#define ADC_MODULE_0          (0U)
#define ADC_MODULE_1          (1U)

typedef enum ADCChannel
{
    ADC_CHANNEL_0              = 0,
    ADC_CHANNEL_1              = 1,
    ADC_CHANNEL_2              = 2,
    ADC_CHANNEL_3              = 3,
    ADC_CHANNEL_4              = 4,
    ADC_CHANNEL_5              = 5,
    ADC_CHANNEL_6              = 6,
    ADC_CHANNEL_7              = 7,
    ADC_CHANNEL_8              = 8,
    ADC_CHANNEL_9              = 9,
    ADC_CHANNEL_10             = 10,
    ADC_CHANNEL_11             = 11,
    ADC_CHANNEL_12             = 12,
    ADC_CHANNEL_13             = 13,
    ADC_CHANNEL_MAX            = 14
} ADCChannel_t;
```

AD03핀을 사용할 것이기 때문에, Module은 0, Channel은 3을 사용
(ex, AD14핀 사용은 Module 1, Channel 4 사용)

Led는 1번핀인 GPIO_GPB(25) 사용

ADC Code – pot_led.h

~/topst-vcp/sources/app.sample/test.app.adc/pot_led.h

```
#ifndef MCU_BSP_POT_LED_H
#define MCU_BSP_POT_LED_H

void Potentiometer_LED_Run(void); // main task에서 호출할 진입 함수

#endif
```

ADC Code – pot_led.c

~/topst-vcp/sources/app.sample/test.app.adc/pot_led.c

```
#include <stdint.h>
#include <gpio.h>
#include <adc.h>
#include "pot_led.h"

#define ADC_TYPE      0
#define ADC_MODULE    0
#define ADC_CHANNEL    ADC_CHANNEL_3    // 가변저항 입력

static const uint32_t led_pin=GPIO_GPB(25);

static void LED_Init(void)
{
    GPIO_Config(led_pin, (GPIO_FUNC(0) | GPIO_OUTPUT)); // 사용할 GPIO_GPB(25)핀 설정
    GPIO_Set(led_pin, 0); // 초기 값 설정
}

void Potentiometer_LED_Run(void)
{
    uint32_t adc_val;

    LED_Init();
    ADC_Init(ADC_TYPE, ADC_MODULE);
    ... 다음 장에 계속
```

ADC Code – pot_led.c

~/topst-vcp/sources/app.sample/test.app.adc/pot_led.c

```
.... 이어서
while (1)
{
    adc_val = ADC_Read(ADC_CHANNEL, ADC_MODULE, 0); // ADC 값 읽기
    adc_val &= 0xFFF; // 하위 12bit 유효값

    if (adc_val > 0 && adc_val < 1000) { // 값이 0~1000이면 led off
        GPIO_Set(led_pin, 0);
    }
    else if (adc_val >= 1000 && adc_val < 2000) { // 값이 1000~2000이면 led on
        GPIO_Set(led_pin, 1);
    }
    else if (adc_val >= 2000 && adc_val < 3000) { // 값이 2000~3000이면 led off
        GPIO_Set(led_pin, 0);
    }
    else if (adc_val >= 3000) { // 값이 3000이상이면 led on
        GPIO_Set(led_pin, 1);
    }
    else {
        GPIO_Set(led_pin, 0);
    }

    mcu_printf("[ADC] ADC=%4d\n", adc_val);
    SAL_TaskSleep(100);
}
}
```

ADC Code – rules.mk / main.c

~/topst-vcp/sources/app.sample/test.app.adc/rules.mk ~/topst-vcp/sources/app.sample/app.base/main.c

```
SRCS += adc_test.c
```

```
// 변경
```

```
SRCS += pot_led.c
```

```
#include "pot_led.h" // 변경
```

```
Static void Main_StartTask(void * pArg)
```

```
{
```

```
    (void)pArg;
```

```
    (void)SAL_OsInitFuncs();
```

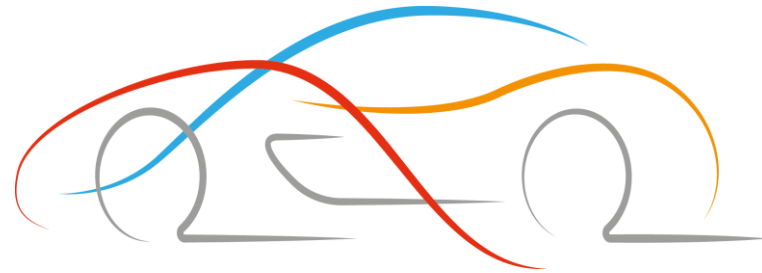
```
    // 변경
```

```
    Potentiometer_LED_Run();
```

```
}
```


Build

- build 진행
 - `$ cd ~/topst-vcp/build/tcc70xx/gcc`
 - `$ make clean && make`
- Build 완료 후 FWDN (빌드한 디렉토리에서 바로 실행)
 - `$ sudo ~/topst-vcp/tools/fwdn_vcp/fwdn`
`--fwdn ~/topst-vcp/tools/fwdn_vcp/vcp_fwdn.rom`
`-w ~/topst-vcp/build/tcc70xx/gcc/output/tcc70xx_pflash_boot_2M_ECC.rom`
- `minicom -D /dev/ttyUSB0 -b 115200 -8` 실행 후 보드 재부팅 시 UART 로그에 가변저항 값 출력.
- 가변저항 값(1000단위)에 따라 led on/off



Thank You